

SafePlus

Experiment 02 阶段性实验报告

Fraud-Onset-Only Baseline: LSTM / GRU 对照、群体分析与多随机种子验证

实验分支: exp/02-fraud-onset-baseline

阶段结论: 本实验把 Alex 草稿第 2.2 节“只建模欺诈实施时间 t_f ”的思路落到了 synthetic 数据上。LSTM-SAFE 与 GRU-SAFE 都能让报警时间明显提前，但群体分析发现：正常用户也大量报警，而且相当一部分报警发生在真实 t_f 之前。5 个随机种子重复实验后，这种“报警偏早、误报偏高”的倾向仍然存在，但不同 seed 的强度有波动。

一、实验背景：从只学 t_d 到尝试学习 t_f

Experiment 01 只建模平台检测时间 t_d ，结果说明模型能够学习“什么时候会被平台检测”，但这不等于知道“欺诈从什么时候真正开始”。因此 Experiment 02 往前推进一步：把目标从 detection time 转到 fraud commission time，也就是 t_f 。

这一步的关键困难是：真实场景中的 t_f 通常不可直接观察。本次 synthetic 数据虽然在生成阶段保留了 `t_onset`，但训练时不把它作为监督输入；只有实验结束后，我们才把 `t_onset` 当作“标准答案”拿出来检查模型到底有没有找准 t_f 。

二、Alex 草稿第 2.2 节：只有范围监督，没有真实 t_f 标签

Alex 在第 2.2 节讨论了 SAFE 的 fraud-onset-only 思路。直觉很简单：既然真实 t_f 看不到，就先利用已经观测到的 detection 信息给 t_f 一个范围。检测到的用户只知道 t_f 一定发生在 t_d 之前；没有检测到的用户则按 $t_f > T$ 处理。

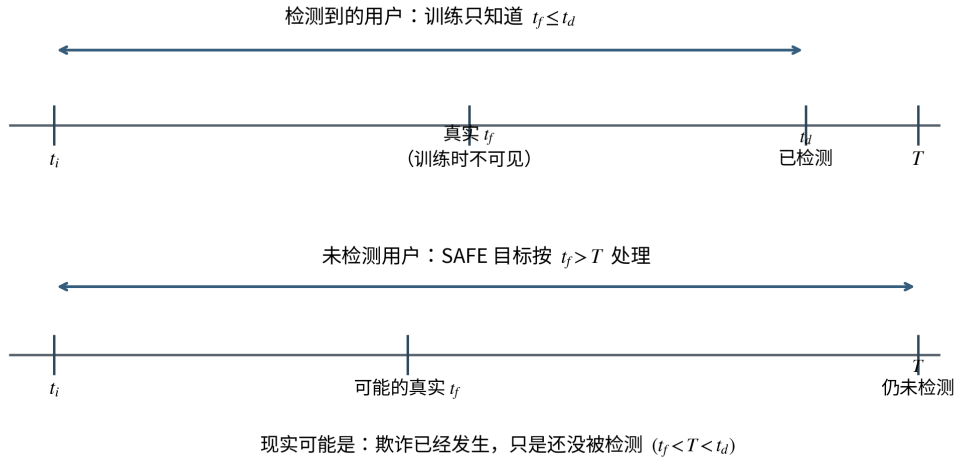


图 1 Experiment 02 的弱监督逻辑

$$\mathcal{L} = \sum_{c_i^d=0} \left[-\log \Pr(t_i^f > T \mid x_i) \right] - \sum_{c_i^d=1} \log \Pr(t_i^f \leq t_i^d \mid x_i). \quad (1)$$

式 (1) Fraud-onset-only 的负对数似然写法（对应草稿第 2.2 节）

Alex 随后自己指出，这个式子仍不完整：一是“没检测到”不代表“没欺诈”，现实可能出现 $t_f < T < t_d$ ；二是 t_d 本身还包含检测延迟信息，例如某类欺诈必须经过一段时间后才可能被发现。我们的实验在此基础上再追问一步：如果正样本只受到 $t_f \leq t_d$ 的范围约束，模型会不会把 t_f 猜得过早？

三、实验设置：LSTM 与 GRU 只换编码器，其他条件保持一致

为了区分“编码器能力”和“训练目标”这两个因素，本实验使用 LSTM-SAFE 和 GRU-SAFE 两个版本。两者都使用同一套 SAFE loss，只更换序列编码器。

实验配置

配置项	设置
数据	data/processed/synthetic_demo.npz
数据规模	$2000 \times 24 \times 8$
Train / Val / Test	0.7 / 0.1 / 0.2 (1400 / 200 / 400)
Hidden dim	32
Batch size	64
Learning rate	0.001
Epochs / patience	40 / 7
模型	LSTM-SAFE / GRU-SAFE
Seeds	0-4

seed=0 的测试集共有 400 个用户。为了看清模型到底在对谁报警，我们把测试集进一步分成三类：265 个正常用户、96 个已欺诈且已检测用户、39 个已经欺诈但观察期内仍未检测的用户。

四、seed=0 基础结果：Recall 很高，但整体区分能力不好

指标	LSTM-SAFE	GRU-SAFE	通俗理解
Accuracy	0.2425	0.2450	全体用户总体判断对不对
AUROC	0.4868	0.5217	风险排序能力；0.5 左右接近随机
AUPRC	0.2391	0.2672	少数类任务下的综合区分能力
Precision	0.2393	0.2412	报警的人里有多少是真的
Recall	0.9896	1.0000	真正正样本有多少被报出来
F1	0.3854	0.3887	Precision 与 Recall 的折中
Early detected rate	0.7708	0.7917	在 t_d 之前报警的正样本比例
Mean lead time	5.25	5.69	平均比 t_d 早多少步报警

如果只看 Recall、Early detected rate 和 Mean lead time，会觉得模型“很会提前发现”。但 Precision 只有约 0.24，AUROC 又接近 0.5，说明这种提前报警并没有建立

在很好的区分能力上。以 GRU 为例，Recall=1.0 表示 96 个 detected 正样本全部被报出；Precision $\approx$ 0.2412，则模型大约报警 $96/0.2412 \approx 398$ 人——也就是说，400 个测试用户里几乎所有人都被报了。

五、群体分析：正常用户也几乎全部报警

这一部分是 Experiment 02 最关键的中间分析。基础指标只能告诉我们“整体表现怎么样”，群体分析才能回答“模型到底对谁在报警”。

模型 / 人群	人数	平均 final risk	报警率
LSTM / 正常	265	0.8071	99.62%
LSTM / 已欺诈 + 已检测	96	0.8070	98.96%
LSTM / 已欺诈 + 未检测	39	0.8084	97.44%
GRU / 正常	265	0.7037	99.25%
GRU / 已欺诈 + 已检测	96	0.7066	100.00%
GRU / 已欺诈 + 未检测	39	0.7031	100.00%

从 final risk 看，LSTM 的三类用户都在 0.807 左右，GRU 的三类用户也都在 0.70 左右，正常人与 fraud 用户并没有明显拉开。更直接的是报警率：LSTM 对正常用户的报警率为 99.62%，GRU 也达到 99.25%。这解释了为什么 Recall 很高、Precision 却很低。

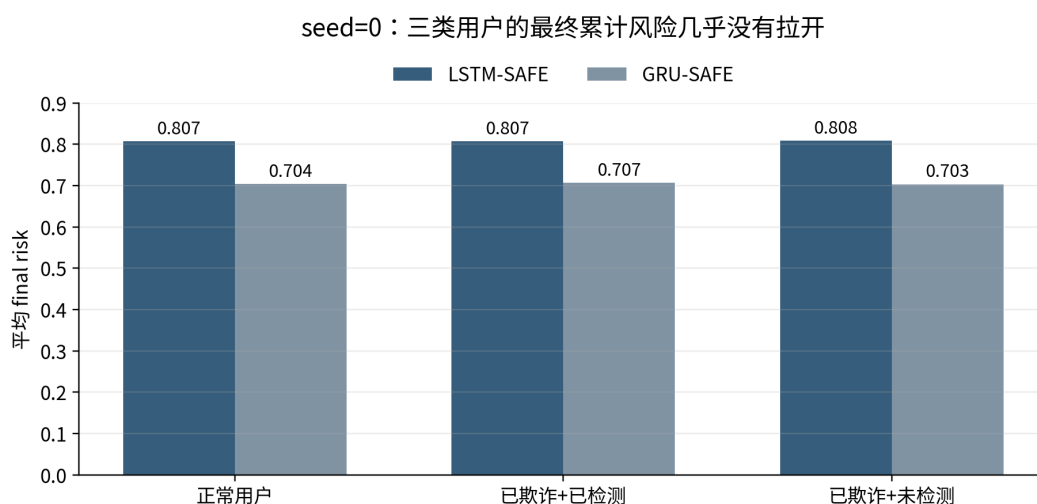


图 2 seed=0：三类用户最终累计风险对比

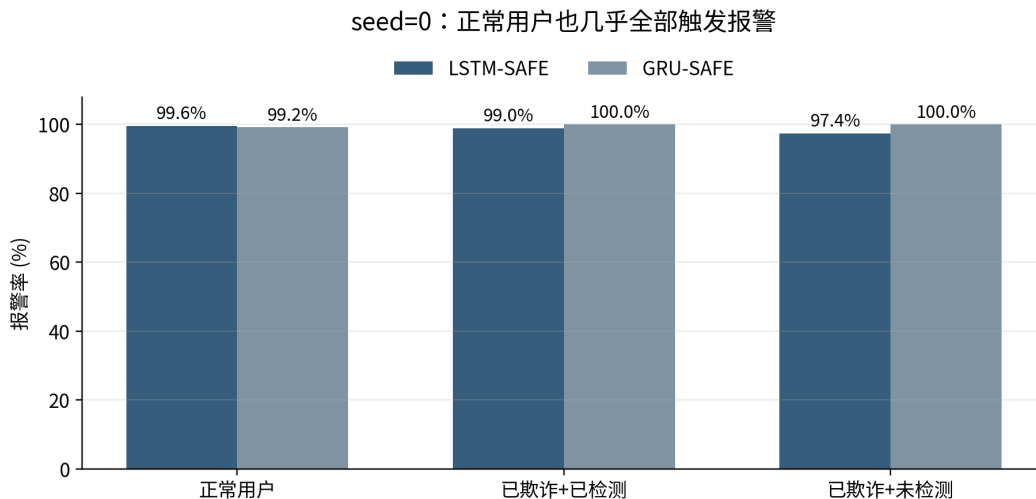


图 3 seed=0：三类用户报警率对比

```
=====
Test-set groups
=====
A. Normal: 265
B. Detected fraud: 96
C. Censored fraud: 39

LSTM-SAFE threshold: 0.7350

-----
A. NORMAL
count: 265
mean final risk: 0.8071
median final risk: 0.8069
min final risk: 0.7325
max final risk: 0.8610
alarm rate: 0.9962

-----
B. DETECTED FRAUD
count: 96
mean final risk: 0.8070
median final risk: 0.8047
min final risk: 0.7314
max final risk: 0.8579
alarm rate: 0.9896

-----
C. CENSORED FRAUD
count: 39
mean final risk: 0.8084
median final risk: 0.8064
min final risk: 0.7279
max final risk: 0.8573
alarm rate: 0.9744
=====
```

图 4 LSTM-SAFE seed=0 群体分析终端输出

六、和真实 t_f 对齐：很多报警其实早过头了

synthetic 数据的优势是我们知道真实 t_{onset} ，因此可以在训练完成后直接比较 first alarm 与真实 t_f 。如果 first alarm 落在 t_f 前面，说明模型不是“早发现了欺诈”，而是在欺诈真正发生前就报警了。

$$\Delta_i = \hat{t}_i - t_i^f, \quad \Delta_i < 0 \iff \hat{t}_i < t_i^f. \quad (2)$$

式 (2) 报警时间相对真实 t_f 的偏移定义

模型 / 群体	t_f 前报警	正好 t_f	t_f 后报警	未报警	平均 $\hat{t} - t_f$
LSTM / Detected	56 / 96 (58.3%)	9	30	1	-1.78
LSTM / Censored	29 / 39 (74.4%)	2	7	1	-3.39
GRU / Detected	63 / 96 (65.6%)	6	27	0	-2.24
GRU / Censored	25 / 39 (64.1%)	2	12	0	-2.38

seed=0 下, 两种编码器都出现了大量 $\hat{t} < t_f$ 。比如 GRU 的 detected fraud 中, 63/96 (65.6%) 在真实 t_f 之前就已经报警; 平均 $\hat{t} - t_f$ 为 -2.24。负数越明显, 说明整体报警越早于真实欺诈发生时间。

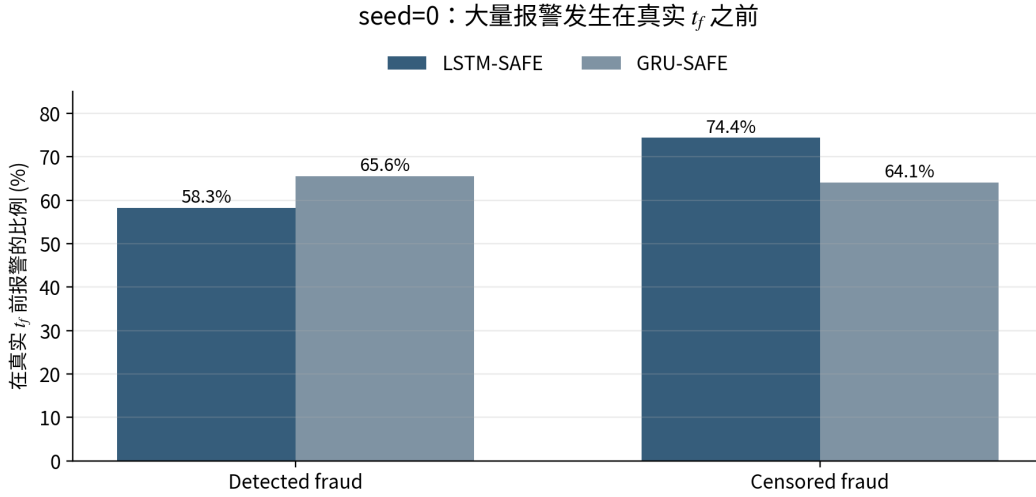


图 5 seed=0: 在真实 t_f 前报警的比例

```

=====
B. Detected fraud vs true tf
=====
mean risk at true tf: 0.7528
median risk at true tf: 0.7487
alarm before true tf: 56
alarm exactly at true tf: 9
alarm after true tf: 30
no alarm: 1
mean(first_alarm - tf): -1.7789

=====
C. Censored fraud vs true tf
=====
mean risk at true tf: 0.7570
median risk at true tf: 0.7562
alarm before true tf: 29
alarm exactly at true tf: 2
alarm after true tf: 7
no alarm: 1
mean(first_alarm - tf): -3.3947

```

图 6 LSTM-SAFE: Detected / Censored 用户与真实 t_f 的对齐统计

因此，Mean Lead Time 很大并不能单独说明模型真正更早识别了欺诈。假设真实 $t_f = 8$ 、平台 $t_d = 15$ ，而模型在 $t = 3$ 就报警，那么相对 t_d 看是“提前 12 步”，但相对真实 t_f 看其实是“提前 5 步误报”。这就是本实验里 early detection 可能虚高的原因。

七、5 个 seed：现象不是 seed=0 的偶然，但强度会波动

为了排除“是不是 seed=0 恰好跑歪了”，LSTM-SAFE 与 GRU-SAFE 分别运行 seeds 0-4，共 10 次训练。每次都重新预测，并重新计算正常用户报警率、 t_f 前报警比例以及 $\hat{t} - t_f$ 。

5-seed 指标	LSTM-SAFE	GRU-SAFE
正常用户报警率	0.8932 ± 0.2220	0.9690 ± 0.0632
Detected: t_f 前报警率	0.7271 ± 0.3434	0.8237 ± 0.2509
Detected: $\text{mean}(\hat{t} - t_f)$	-4.6244 ± 4.5021	-5.9308 ± 4.3248
Censored: t_f 前报警率	0.7676 ± 0.3134	0.8110 ± 0.2529
Censored: $\text{mean}(\hat{t} - t_f)$	-5.8278 ± 2.9574	-5.8810 ± 4.6102
AUROC	0.5035 ± 0.0243	0.5108 ± 0.0395
AUPRC	0.2627 ± 0.0293	0.2704 ± 0.0355
Precision	0.2505 ± 0.0173	0.2497 ± 0.0179
Recall	0.8907 ± 0.2335	0.9634 ± 0.0817

多 seed 结果说明：LSTM 和 GRU 都重复出现了“正常人报警率高、 t_f 前报警比例高、平均 $\hat{t} - t_f$ 为负”的现象，因此它不像是某一种编码器特有的问题。不过不同 seed 的强度差别很大，例如 GRU seed=3 明显比 seed=1、2、4 温和，所以这里应该表述为“明显的经验性倾向”，而不是说它必然发生。

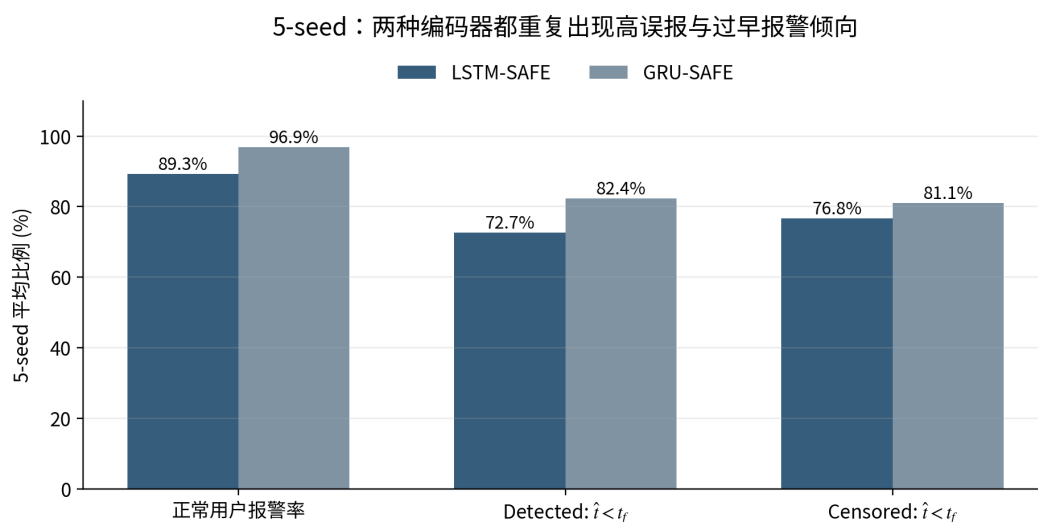


图 7 5-seed：关键比例汇总

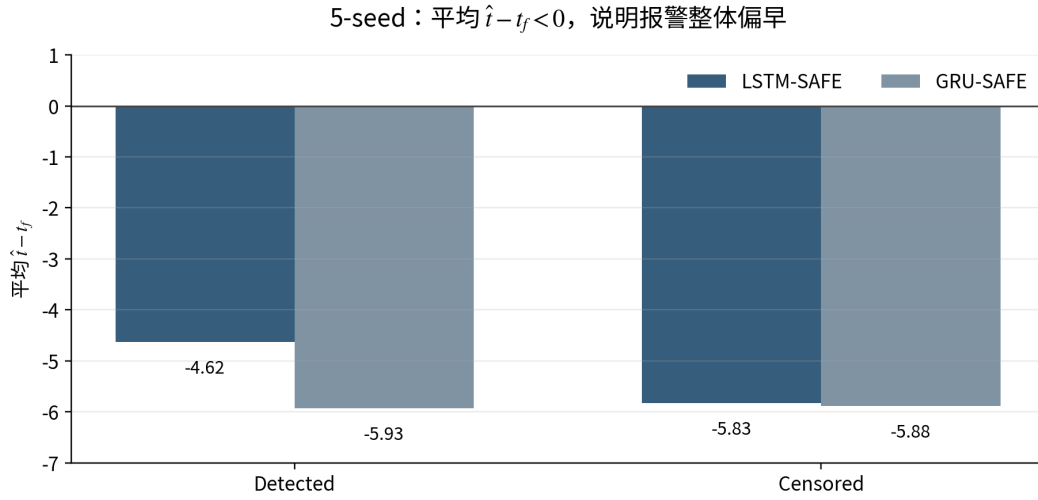


图 8 5-seed: 报警时间相对真实 t_f 的平均偏移

```
GRU-SAFE | seed=1
normal alarm rate: 1.0000
detected before tf rate: 1.0000
detected mean(alarm-tf): -8.9065
censored before tf rate: 1.0000
censored mean(alarm-tf): -8.1852

>>> GRU-SAFE | seed=2

=====
RUN:
C:\Users\28405\miniconda3\python.exe -m safeplus train --config F:\research\SafePlus\conf
nfigs\synthetic\exp02_2_r3rplpb7.yaml
=====
runs\synthetic\exp02\gru-safe\seed_2\metrics.json

=====
RUN:
C:\Users\28405\miniconda3\python.exe -m safeplus predict --run-dir F:\research\SafePlus
\runs\synthetic\exp02\gru-safe\seed_2 --data F:\research\SafePlus\data\processed\synthe
tic_demo.npz --output F:\research\SafePlus\runs\synthetic\exp02\gru-safe\seed_2\predict
ions.npz --batch-size 64 --device cpu
=====
F:\research\SafePlus\runs\synthetic\exp02\gru-safe\seed_2\predictions.npz

=====
GRU-SAFE | seed=2
normal alarm rate: 0.9963
detected before tf rate: 1.0000
detected mean(alarm-tf): -8.9200
censored before tf rate: 0.9655
censored mean(alarm-tf): -8.6552

>>> GRU-SAFE | seed=3
```

图 9 多随机种子运行过程节选 (GRU-SAFE seed=1/2)

八、和 Alex 草稿对照：原来已经知道什么，我们补充了什么

对照点	Alex 草稿第 2.2 节	本次实验补充
未检测用户	已经指出：可能存在 $t_f < T < t_d$ ，也就是欺诈已发生但仍未检测。	测试集确实有 39 个这类用户，说明这种情况在受控数据中真实存在。
检测时间信息	已经指出： t_d 本身还包含检测延迟信号，单独建模 t_f 会丢掉这部分信息。	Experiment 02 暂时不解决，下一步 joint model 再验证。
t_f 的定位	草稿主要要求 t_f 应发生在 t_d 之前，没有展开“会不会早过头”。	LSTM / GRU 与 5-seed 实验都出现大量 $\hat{t} < t_f$ 。
Early Detection	目标是把报警从 t_d 向前拉。	实验显示：只看相对 t_d 的提前量可能虚高，因为报警可能已经早于真实 t_f 。

Alex 已经凭直觉指出单 t_f 模型不完整；我们做的是把这个直觉放到有真实 t_f 的 synthetic 数据里，并补充看到一个更具体的实验现象：只有 $t_f \leq t_d$ 这种范围监督时，报警时间并没有被稳定锁在真实 t_f 附近，甚至会出现大量过早报警。

九、阶段性结论与下一步

- Experiment 02 已完成 LSTM-SAFE、GRU-SAFE、seed=0 群体分析、真实 t_f 对齐以及 5-seed 稳定性验证，形成完整闭环。
- Alex 已指出的“未检测 $\neq$ 未欺诈”在测试集中真实存在：39 个用户已经 fraud，但观察期内仍未检测。
- 本实验进一步观察到：只靠 $t_f \leq t_d$ 这类范围监督时，模型的报警时间没有稳定落在真实 t_f 附近；两种编码器都出现大量 $\hat{t} < t_f$ 。
- 因此，高 Recall、高 Early Detected Rate 和较大的 Mean Lead Time 不能单独当作“找准 t_f ”的证据，还要同时检查 Precision、AUROC/AUPRC 以及 $\hat{t} - t_f$ 。
- 不同 seed 的结果有明显波动，因此目前应把“过早报警”写成经验性倾向，而不是普遍定理。
- 下一步进入 Experiment 03：同时建模 $\lambda_f(t)$ 与 $\lambda_d(t)$ ，检验 joint model 是否能利用 detection delay 信息，把报警时间更稳定地拉回真实 t_f 附近。

十、可复现记录

实验分支为 `exp/02-fraud-onset-baseline`。主要产物包括 LSTM-SAFE / GRU-SAFE 实现与配置、`seed=0` 三类用户分析脚本、`5-seed` 汇总脚本,以及 `reports/experiment02/` 下的 `exp02_multiseed_results.csv` 与 `exp02_multiseed_summary.json`。

参考资料

Alex Smola. *Survival Analysis for Fraud Detection*. Technical note, 2022.

Panpan Zheng, Shuhan Yuan, Xintao Wu. *SAFE: A Neural Survival Analysis Model for Fraud Early Detection*. AAAI, 2019.